

Experiment No: 03

Machine Learning

Aim:

Implement k-NN from scratch in Python to classify Iris flowers, varying k to observe accuracy changes.

Objective:

- To understand the working of k-Nearest Neighbors (k-NN) classification algorithm.
- To implement k-NN without using sklearn's built-in classifier.
- To experiment with different values of k and analyze its impact on accuracy.
- To visualize accuracy vs k through a plot.

Prerequisites:

- Software: Python 3.x installed with pandas, numpy, matplotlib, scikit-learn.
- Hardware: A computer with minimum 4GB RAM.
- Skills: Basic Python programming and understanding of supervised learning concepts.

Theory:

The k-Nearest Neighbors (k-NN) algorithm is a non-parametric, instance-based supervised learning method.

It classifies a data point based on the majority class of its nearest neighbors in the feature space.

The distance metric (commonly Euclidean) is used to find the closest neighbors.

The choice of k significantly impacts performance: small k may cause overfitting, while large k may oversmooth decision boundaries.

This experiment demonstrates the implementation of k-NN from scratch and evaluates accuracy for different values of k.

Apparatus/Software Requirements:

- Python IDE or Jupyter Notebook.
- Internet connection (for library installation).
- Cars_Datasets_2025.csv dataset.

Procedure:

1. Step 1: Load the dataset (Cars_Datasets_2025.csv) into a pandas DataFrame.
2. Step 2: Encode categorical features into numerical values using LabelEncoder.
3. Step 3: Split the dataset into training and testing sets (70-30 split).
4. Step 4: Implement the k-NN algorithm from scratch using Euclidean distance.
5. Step 5: For each test sample, calculate distances from all training samples, pick k nearest neighbors, and assign the majority class.
6. Step 6: Test the classifier for different values of k (1 to 20).

7. Step 7: Calculate and print accuracy for each value of k.
8. Step 8: Plot accuracy vs k to visualize performance changes.

Program Code:

```
import pandas as pd
import numpy as np
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

df = pd.read_csv("Cars_Datasets_2025.csv", encoding="latin1")

le_dict = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    le_dict[col] = le

X = df.drop(columns=["Fuel Types"]).values
y = df["Fuel Types"].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        return np.array([self._predict(x) for x in X])

    def _predict(self, x):
        distances = [np.sqrt(np.sum((x - x_train) ** 2)) for x_train in
                    self.X_train]
        k_idx = np.argsort(distances)[:self.k]
        k_neighbor_labels = [self.y_train[i] for i in k_idx]
        most_common = Counter(k_neighbor_labels).most_common(1)
```

```
return most_common[0][0]

accuracies = []
k_values = range(1, 21)

for k in k_values:
    model = KNN(k=k)
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = np.mean(preds == y_test)
    accuracies.append(acc)
    print(f"k = {k}, Accuracy = {acc:.3f}")

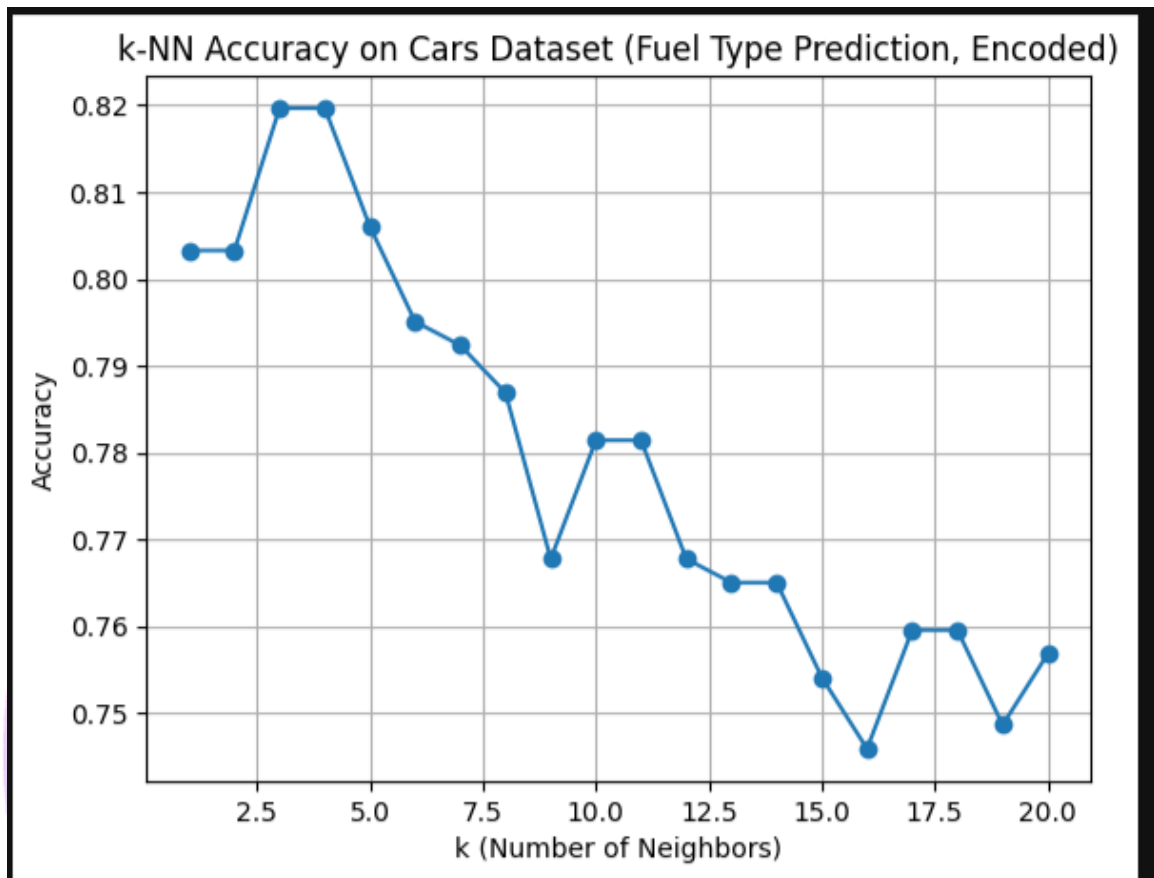
plt.plot(k_values, accuracies, marker="o")
plt.xlabel("k (Number of Neighbors)")
plt.ylabel("Accuracy")
plt.title("k-NN Accuracy on Cars Dataset (Fuel Type Prediction, Encoded)")
plt.grid(True)
plt.show()
```

Expected Output:

- Display of accuracy values for k = 1 to 20.

k = 1, Accuracy = 0.803
k = 2, Accuracy = 0.803
k = 3, Accuracy = 0.820
k = 4, Accuracy = 0.820
k = 5, Accuracy = 0.806
k = 6, Accuracy = 0.795
k = 7, Accuracy = 0.792
k = 8, Accuracy = 0.787
k = 9, Accuracy = 0.768
k = 10, Accuracy = 0.781
k = 11, Accuracy = 0.781
k = 12, Accuracy = 0.768
k = 13, Accuracy = 0.765
k = 14, Accuracy = 0.765
k = 15, Accuracy = 0.754
k = 16, Accuracy = 0.746
k = 17, Accuracy = 0.760
k = 18, Accuracy = 0.760
k = 19, Accuracy = 0.749
k = 20, Accuracy = 0.757

- Plot showing accuracy vs k.



- Best accuracy at some optimal k value.

Observations:

The classifier's accuracy varied with k. Lower k values produced high variance results, while larger k values provided smoother but sometimes less accurate predictions. An optimal k was observed for best accuracy.

Conclusion:

This experiment successfully implemented the k-NN classifier from scratch and analyzed its performance on the Cars dataset. The impact of k on classification accuracy was visualized, demonstrating the trade-off between bias and variance.

Viva Questions:

1. What is the role of k in the k-NN algorithm?
2. Why do we use Euclidean distance in k-NN?

3. How does the choice of k affect model performance?
4. What are the advantages and limitations of k -NN?
5. How can feature scaling impact k -NN results?

